Goal-conditioned dual-action imitation learning for dexterous dual-arm robot manipulation

Heecheol Kim[1,2], Yoshiyuki Ohmura[1], Yasuo Kuniyoshi[1]

arXiv:2203.09749v3 [cs.RO] 21 May 2025

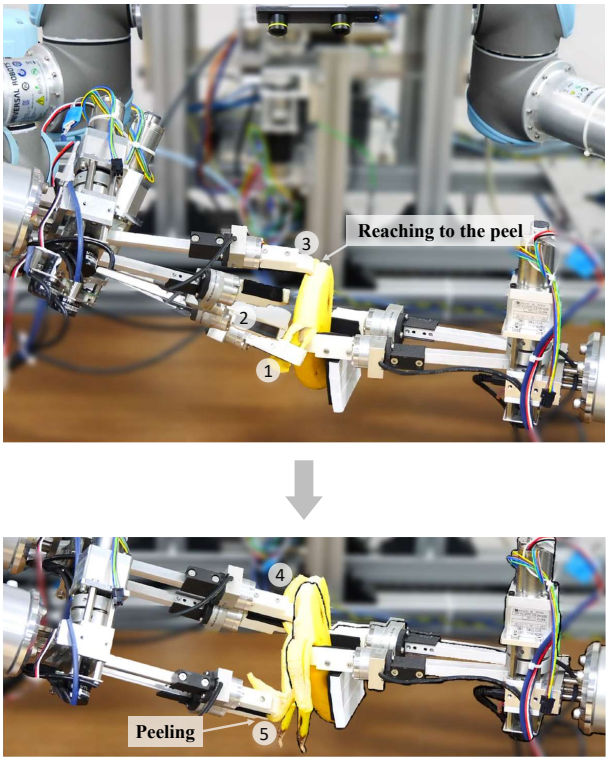
***Abstract*—Long-horizon dexterous robot manipulation of deformable objects, such as banana peeling, is a problematic task because of the difficulties in object modeling and a lack of knowledge about stable and dexterous manipulation skills. This paper presents a goal-conditioned dual-action (GC-DA) deep imitation learning (DIL) approach that can learn dexterous manipulation skills using human demonstration data. Previous DIL methods map the current sensory input and reactive action, which often fails because of compounding errors in imitation learning caused by the recurrent computation of actions. The method predicts reactive action only when the precise manipulation of the target object is required (local action) and generates the entire trajectory when precise manipulation is not required (global action). This dual-action formulation effectively prevents compounding error in the imitation learning using the trajectory-based global action while responding to unexpected changes in the target object during the reactive local action. The proposed method was tested in a real dual-arm robot and successfully accomplished the banana-peeling task.**

***Index Terms*—Learning from Demonstration, Dexterous Manipulation, Deep Learning in Robotics and Automation, Dual Arm Manipulation**

I. Introduction

DEEP imitation learning, which trains the robot's behavior using human-generated demonstration data with deep neural networks, is a promising technique because it can transfer the implicit human knowledge of dexterous manipulation into a robot system without a pre-defined manipulation rule based on knowledge of the objects [1]–[4].

This paper presents a deep imitation learning approach to achieve dexterous robot manipulation tasks such as banana peeling (Fig. 1) with a general-purpose dual-arm robot manipulator (UR5, Universal Robots). In this task, every banana's shape, size, and skin pattern are different (Fig. 1b), making banana model acquisition difficult. In addition, the deformation of the banana during peeling makes this task even more difficult. The diversity in the sensory input state, brought about by the varying nature of the banana, necessitates high precision in the robot's actions to ensure the policy remains within the permitted action space. For example, because the banana is deformable, soft, and easily breakable, the robot has to accurately position its finger on the banana's peel and tear it away while maintaining its body; this requires highly dexterous and precise manipulation skills that consider physical interactions with the target object, which are difficult for rule-based robotics to acquire. Moreover, the banana peeling is a difficult task that consists of a combination of several reaching and peeling subtasks and requires stable manipulation policies for consecutive success on each subtask. Therefore, to our knowledge, no previous robotics research has tackled this task.



(a) Part of the automated banana peeling sequence in which the robot extends its end-effector to the banana (1–2) and peels it (3–4).



(b) Example bananas used in the evaluation.

Fig. 1. Proposed goal-conditioned dual-action *(GC-DA)* imitation learning (a) can peel real bananas of various shapes (b) using a general-purpose dual-arm robot.

Our previous research proposed a visual attention-based deep imitation learning framework [5] that can focus on an important area of visual input by using the gaze signal of

[1] Laboratory for Intelligent Systems and Informatics, Graduate School of Information Science and Technology, The University of Tokyo, 7-3-1 Hongo, Bunkyo-ku, Tokyo, Japan (e-mail: {h-kim, ohmura, kuniyosh}@isi.imi.i.u-tokyo.ac.jp, Fax: +81-3-5841-6314)

[2] Corresponding author

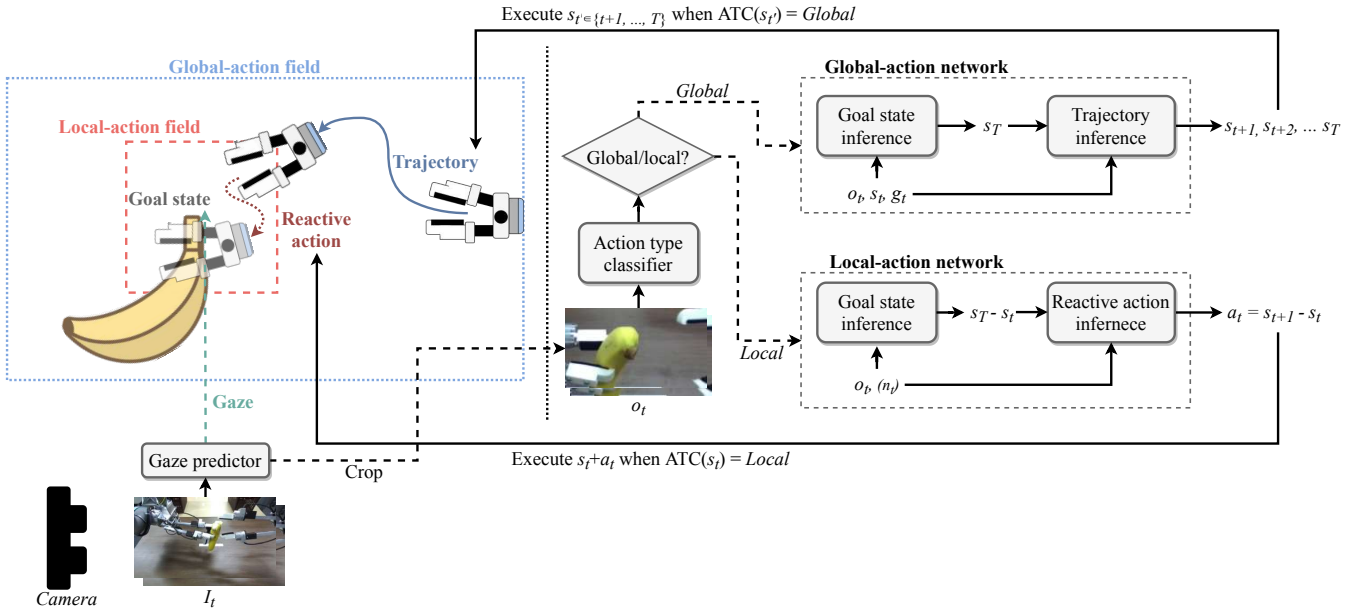


Fig. 2. Proposed *GC-DA* architecture. The action-type classifier (ATC) determines if precise manipulation of the target object is required; if not, the global-action network computes the trajectory of the end-effector, which is robust against compounding errors; otherwise, the local-action network computes a reactive action that can dexterously manipulate the target object. In practice, once the ATC's output transitions from local-action to global-action or global-action to local-action, the ATC's output is fixed, improving system robustness despite possibly noisy output of the ATC. Here, o_t, n_t, s_t, g_t, a_t, and T refer to the foveated image, force/torque sensory values, robot kinematics states, gaze position, action, and last timestep in the sequence, respectively. Detailed neural network architectures for the global-action network, local-action network, and gaze-predictor can be found in Fig. 4.

a human expert while demonstrating the task. Subsequently, [3] proposed a dual-resolution dual-action system that handles local action in the vicinity of the gaze position separately from global action outside of the gazed area. This approach has successfully completed dexterous manipulation tasks such as needle threading. However, the dual-resolution dual-action system was applied to tasks in which the objects are located in two-dimensional space, which is narrower than the banana-peeling task. Moreover, the state variation of the needle is narrower because the needle is not deformable. Therefore, the previous dual-resolution dual-action system cannot provide data efficiency and a stable trajectory for the banana-peeling task.

The problems of the current dual-action system described above arise from the compounding error caused by reactive action. In the previous dual-action system, both global and local actions are a reactive action, which is the difference between the state in the next step and the current state, that is, $a_t = s_{t+1} - s_t$, where a_t is the reactive action and s_t is the robot kinematic state. This reactive action can respond to unexpected immediate changes in the environment caused by an incomplete environment model and error in policy predictions. However, because the reactive action changes the robot state recurrently, in long-horizon tasks, the state eventually escapes from the training distribution because of compounding error in the action prediction. Moreover, if the agent predicts the entire trajectory, the problems caused by the compounding error occur less frequently, but the trajectory-based manipulation cannot deal with immediate changes in the environment.

We hypothesize that in the manipulation task, the immediate changes in the environment usually occur while the end-effector is interacting with the target object. Therefore, if the end-effector is in the vicinity of the object, reactive action is required; if it is not, trajectory-based action can stably transport the robot toward the goal position.

On the basis of the hypothesis described above, we propose a method that uses reactive action for dexterous manipulation and trajectory-based action for robustness against compounding error (Fig. 2). This architecture separately handles local action in the sequence of the robot behavior, where the local action is defined as the reactive action output from the visually attended feature input images using robot gaze (foveated images). The global action is a trajectory $s_{t+1}, s_{t+2}, \ldots, s_T$, where T indicates the last step in the sequence, and is computed from the foveated image, current robot kinematic state, and gaze position. A simple CNN-based classifier can switch the action between global and local action based on a criterion that considers whether precise control of the target object is required.

This research also improves manipulation performance by predicting the goal state, which is defined as the robot's kinematic state at the last timestep of each subtask, and the goal-conditioned action inference. Previous deep imitation learning methods predicted reactive action from the input states of the current timestep [1], [2], [5]. However, action should also be conditioned by the goal state because the action can be considered a part of the route from the current state to the goal state; therefore, the action must be changed if the goal state is changed. A neural network only statistically maps the action from the current sensory input state without goal conditioning, which is fragile to distractions in the input state.

Therefore, the proposed model first predicts the goal state, which is defined as the last state in the sequence, and then the action is predicted using the goal state and the current sensory input state to achieve robust policy prediction.

The proposed goal-conditioned dual-action (*GC-DA*) imitation learning model was tested with the banana-peeling task in a real dual-arm robot manipulator (UR5).

To summarize, our contributions are as follows:

- A dual-action architecture that is a combination of reactive local action and trajectory-based global action to achieve dexterity while maintaining stability in the manipulation policy.
- Goal-based policy inference using the dual-action framework.
- Evaluation of the proposed deep imitation learning framework using the challenging banana-peeling task with a real-world dual-arm robot.

II. Related work

A. Trajectory-based methods for robot manipulation

Trajectory generation methods have been studied widely in robotics (e.g., [6], [7]). For example, [8] studied searching trajectories in reinforcement learning using a factorized-action variational autoencoder [9]. Notably, in path planning methods in which a robot plans its path to a goal while avoiding obstacles, global path planning and local path planning are present. Global path planning plans the entire path using global information [10]–[13]. By contrast, the local path planning computes real-time reactive action based on local information and is robust to unpredicted disturbances in the environment [14]–[17]. Hierarchical planners that combine global and local path planning have achieved collision-free motion without delays caused by the local path planner [18]. This approach supports our system, which combines trajectory-based global action and reactive local action. Nevertheless, these path planning-based methods cannot be directly applied to our target task because acquiring a deformable and high-variance object model from visual sensory input is difficult, and both the path planning and reactive action with the target object are unknown.

B. Goal-based policy learning

Many different aspects of goal-based policy learning have been studied. [19]–[21] considered a goal while searching for a policy during manipulation. [19] proposed hindsight experience replay (HER), which assumes the last state in an explored episode is the goal, even if the episode was not successful. This goal is used for the data-efficient training of a goal-conditioned policy. [20] applied HER to generative adversarial imitation learning [22], which uses generative adversarial training [23] for reward learning to leverage policy convergence speed. [21] acquired dense rewards using a goal proximity function trained using demonstration data. However, exploration-based methods cannot be applied to our task setup, which must train a dexterous robot policy using only a limited number of bananas as training data.

Several studies do not rely on exploration in addition to demonstration data [24]–[31]. [24]–[27], [30], [31] employed the goal to help predict the policy. [24], [25] used user-provided goal scenes to select pre-defined manipulation behavior from an axiomatic scene graph of current visual input. These methods successfully manipulated objects in long-horizon tasks in cluttered scenes. [28], [29] selected high-level action using goal-based imitation, where the low-level actions are pre-defined. [26], [27] inferred the primitive action from the goal. [26] trained a latent variable model using a variational autoencoder [32] to control a robot manipulator toward the given goal. [27] proposed a neural network architecture conditioned on high-level commands for autonomous driving. In these methods, the goal is used as a signal to change the agent's behavior. By contrast, in our proposed method, the goal prediction and conditioning are employed to achieve a stable and dexterous manipulation policy.

[2] improved manipulation accuracy by training the auxiliary prediction, which estimates the gripper's position when closed during picking. [30] leveraged goals to manage the high variability in demonstration quality and task solution approaches commonly seen in human expert data. [31] utilized the intersecting structure of the training dataset to formulate a policy that can generalize to unseen start and goal state combinations. In contrast, our goal-conditioned approach aims to generate a stable policy for the highly complex task of banana peeling. Our demonstration set, derived from a highly skilled expert, inherently exhibits low variability in subgoals and there are few alternative policies for achieving the subgoals.

C. Automation in food processing

Studies on food processing automation using robots include food manufacturing in factories [33], [34], fruit harvesting [35]–[40], and cooking [41], [42]. Many of these studies relied on task-specific end-effector designs [39], [40], [43], [44] or rule-based manipulation [35]–[37]. Therefore, they lacked the acquisition of dexterous manipulation skills, instead, they limited the target task to simple object manipulations, such as pick-and-place tasks.

Some work has studied long-horizon deformable object (dough) manipulation in a simulator [45] and real-world [46]. However, learning for the banana-peeling task requires highly precise manipulation skills in three-dimensional state space, which requires skills that are substantially different to those required for dough control.

[47], [48] combined an open-loop motion planner and closed-loop neural-network-based policy, leveraging the merits of both methods. However, these methods require a reinforcement-learning-based policy search, which is infeasible in deformable and destructible object manipulation and requires an additional pipeline with an open-loop motion planner. By contrast, the proposed *GC-DA* relies on the imitation learning framework, which is much more sample efficient and requires no additional traditional robot control pipeline but can still leverage the strength of the combined robust open-loop control and precise closed-loop object manipulation.

Some learning-based methods have been applied to food processing as target tasks, such as pouring [49], [50] or stirring

[51]. However, those tasks usually do not require manipulation skills as precise as those needed in our target task of banana peeling; therefore, a new method is needed for learning highly skilled tasks.

Hence, this research uses a general-purpose manipulator with a single degree-of-freedom end-effector, which can also be applied to other various tasks [3]–[5], [52], to learn dexterous manipulation skills such as banana-peeling by imitating human manipulation skills.

III. Preliminaries

A. Gaze-based attention

First proposed in [5], gaze-based attention is a method that acquires visual attention using a human's gaze information, obtained during demonstrations. This method measures the human gaze during a demonstration with an eye tracker and trains the neural network model to output the gaze's probability distribution from the camera's visual input. Using the model, the robot can visually attend to the most probable position in a given visual scene. Using the predicted gaze coordinates, the model crops the foveated vision from the entire raw pixel image, and this foveated vision is used as the visual input to another neural network model, which computes the action policy. An important advantage of using gaze-based attention is that it significantly reduces the number of network parameters and the size of the training dataset for tasks that necessitate high-resolution vision for precise manipulation [3]. Without the gaze model, the neural network would be obligated to process the entire high-resolution vision (1280 x 720 for this task). However, the majority of these pixels contain irrelevant information, such as the background and unrelated objects. This leads to impractical training time and space requirements. Also, Because the task-unrelated background and other objects are masked out in the foveated vision, gaze-based attention is robust against the distractions in the background and task-unrelated objects and is more computationally efficient [5]. Fig. 4 (c) illustrates this model.

B. Dual-action

First proposed in our previous study [3], dual-action is a method that distinguishes actions in the global reaching behavior and precise object manipulation to enable dexterous manipulation skills while maintaining global reaching ability. This method handles the local action, which consists of precise manipulation skills that require interaction with the target object, separately from the global action, which delivers the robot hand to the area near the target. The local-action network is trained only with local action so that it is not affected by disturbances from the global action. In our previous study [3], both the global and local actions are reactive and not conditioned on the goal state. In these studies, the criterion for distinguishing global/local action was based on speed, but this does not apply to general tasks, where a significant difference in speed between local and global actions is not guaranteed. Therefore, we annotated the global/local action manually based on whether the robot's end-effector is in the foveated vision.

IV. Method

A. Robot framework

This research uses a dual-arm robot framework for imitation learning that was also used in our previous studies (e.g., [3], [5]). This framework consists of a dual-arm robot system with two UR5 manipulators and two controllers with identical kinematic parameters with the UR5 robot. Demonstration data are generated by controlling the robots with the controllers. A ZED mini stereo camera (StereoLabs, [53]) is mounted on the robot system with a two-dimensional pan-tilt structure. In this research, the camera is fixed at a position that can observe bananas (pan: 0 rad, tilt: -0.9 rad). The human operator can see the stereo camera image through a head-mounted display while operating the robot. Furthermore, an eye tracker (Tobii) is mounted on the head-mounted display to measure the operator's gaze position in real time.

In this method, s_t is the robot state at timestep t defined by concatenated left and right arm states. Each arm state consists of three dimensions to express the position (x,y,z), six dimensions for the cosine and sine of the Euler angle for rotation, and one dimension for the gripper angle. The Euler angle is decomposed into cosine and sine expressions to enable a smooth expression when the angle exceeds $2 * \pi$. The action of each arm is hence a seven-dimensional vector with the differences in position (three dimensions), Euler angle (three dimensions), and gripper (one dimension) in adjacent timesteps.

B. Task specification

The banana-peeling task can be segmented into several subtasks (Table I). The entire banana-peeling sequence is segmented using natural semantic criteria: the robot first grasps the banana (*GraspBanana*), then picks the banana up (*PickUp*), moves its arm to the banana's neck (*GraspNeck*), peels it (*PeelNeck*), moves its arm to the banana's peel (*ReachRight*), peels it (*PeelRight*), rotates the banana so that the right hand can reach the left peel (*Reposition*), moves the arm to the banana's last peel (*ReachLeft*), and then peels it (*PeelLeft*).

Each subtask consists of a combination of global and local actions. Among all possible combinations of dual-action (global-only, local-only, global-to-local, and local-to-global), the subtask selects one based on the required dexterity. As shown in Fig. 3, each subtask requires different levels of dexterity. For example, *GraspNeck, ReachRight*, and *ReachLeft* do not require high dexterity while moving the robot hand to the vicinity of the neck or peels, but skillful manipulation is required when the hand is near the target and tries to reach the exact position. During peeling subtasks, a closed-loop control is needed until the robot has peeled the target. Once the target has been peeled, less dexterity is required. Therefore, the global-to-local combination is used for grasping and reaching subtasks, and peeling subtasks use local-to-global combinations. Simple subtasks that do not require dexterity (*GraspBanana, PickUp*, and *Reposition*) use global-only action.

A next step classifier (NSC) is a neural network that detects whether the current executing subtask needs to end and to

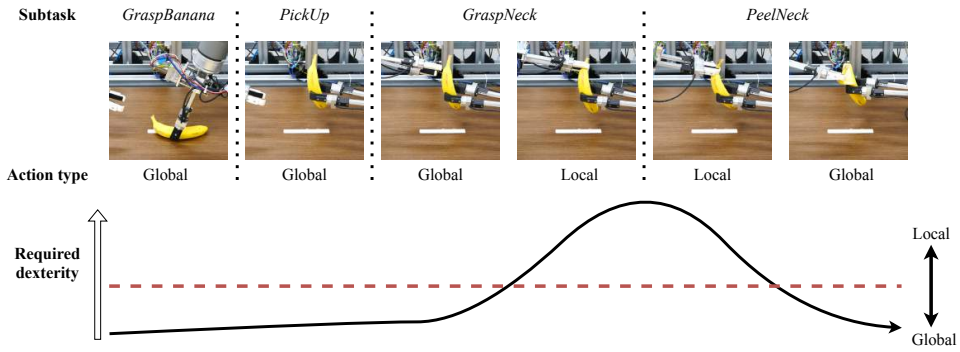


Fig. 3. Relationship between the required dexterity for the subtasks and connection to the action type. Local action is used for the tasks that require more dexterity.

robot should transit to the next subtask. This network is required for subtasks that end with a local action (i.e., *GraspNeck* to *PeelNeck*, *ReachRight* to *PeelRight*, and *ReachLeft* to *PeelLeft*). NSC is composed of a multi-layer perceptron (MLP) architecture followed by a Conv module (Fig. 4). It uses the foveated vision as input at outputs a binary classification result, where the former subtasks (*GraspNeck*, *ReachRight*, and *ReachLeft*) are assigned the label 0 and the latter subtasks (*PeelNeck*, *PeelRight*, and *PeelLeft*) are assigned the label 1.

Another possible strategy for banana peeling is to re-grasp the banana with the right hand and peel the left side with the left hand. However, the banana may become unstable after the re-grasping. Therefore, the banana is rotated rather than re-grasped. In addition, many people peel the banana from the banana's tip. However, the robot was not able to peel from the tip even with the human operator because of a lack of grasping power.

During demonstrations, because the number of bananas was limited, non-destructive subtasks were repeated to maximize the number of demonstrations. *GraspBanana*, *PickUp*, *GraspNeck*, *ReachRight*, *Reposition*, and *ReachLeft* do not tear or break the target banana; therefore, those subtasks were repeated 10–20 times per banana. By contrast, *PeelNeck*, *PeelRight*, and *PeelLeft* peel the bananas; therefore, these subtasks are non-reversible. Thus, these subtasks were recorded only once per banana. All demonstrations were recorded at 5 Hz. Table II presents the statistics of each subtask.

C. Goal-conditioned dual-action

Figure 2 and Algorithm 1 illustrate the concept of goal-conditioned dual action. The dual-action-based architecture separates the reactive local action, which is used when precise control is required, from the trajectory-based global action, which is used when path planning that is robust against compounding error is required.

The boundary between local action and global action is decided by whether the robot requires precise manipulation. This boundary decision is annotated by humans and then trained using a binary classifier to autonomously switch between global and local action (see IV-E for details). Notably, on high-precision *GraspNeck/Reach*$*$ subtasks, a simple criterion, which is whether the end-effector is in the foveated image, can determine the boundary (see Appendix B for details).

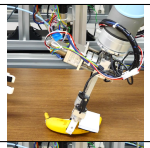
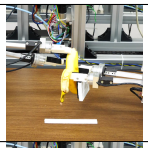
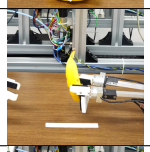
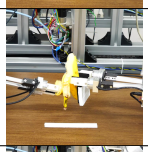
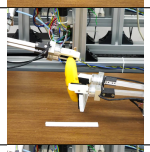
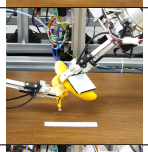
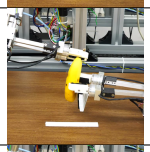
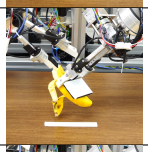
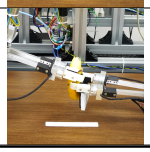
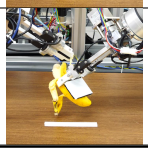
Both global and local actions are conditioned by the goal state, which is the robot kinematic state at the last timestep T of the sequence in each subtask. Simple end-to-end imitation learning $a_t = s_{t+1} - s_t = \pi(e_t)$, where π, e_t, s_t, and a_t respectively refer to the policy, embedding of the current input states, robot kinematic state, and current action, which maps s_t to a_t, is unstable with respect to the distractions in s_t. By contrast, the explicit inferred goal state s_T, where T refers the last timestep in the sequence, can provide a stable policy because the action can use explicit information about the task goal. The trajectory-based global-action network first predicts the goal state: $s_T = \pi^{goal}_{global}(e_t)$, where s_T and π^{goal}_{global} respectively refer to the inferred robot kinematic goal state and the goal predictor of the global-action network. Then, trajectory $s_{t+1}, s_{t+2}, ..., s_{T-1}$ is predicted using both e_t and s_T: $s_{t+1}, ..., s_{T-1} = \pi^{policy}_{global}(e_t, s_T)$, where π^{policy}_{global} refers to the policy inference of the global-action network. For all subtasks except for *GraspBanana*, *PickUp*, and *Reposition*, which are only composed of global action, the predicted trajectory is executed during $ATC(o_{t' \in \{t+1,...,T\}}) = global$, where ATC indicates the action-type classifier.

The action type classifier (ATC) is trained to predict the type of global/local action from the foveated vision as input. The purpose of this network is to predict the transition from global action to local action while executing the policy networks in the real robot environment. The dual-action system establishes a link between foveated vision and local action, an idea grounded in the findings of physiological studies [54]. Consequently, the visual presence of the robot's gripper in the foveated vision serves as a logical criterion for action type identification. This network utilizes the same Conv module architecture (Fig. 4 (d)) and an MLP with two-dimensional output for the global/local labels.

The reactive local-action network only inputs the foveated image o_t because the reactive local action can be computed efficiently from the relation between the target object and the end-effector [3]. In this architecture, information about the global coordinate is lost, and the neural network only focuses on the relationship between the target object and the end-effector. Therefore, this network predicts and uses

TABLE I

SUBTASK DETAILS AND A VISUAL EXAMPLE OF THE GOAL. "ATC" INDICATES WHETHER AN ACTION-TYPE CLASSIFIER (ATC) IS REQUIRED TO DETECT THE TRANSITION FROM A GLOBAL TO LOCAL (OR LOCAL TO GLOBAL) ACTION. "NSC" INDICATES WHETHER THE NEXT STEP CLASSIFIER (NSC) IS REQUIRED TO DETECT THE END OF THE SUBTASK SO THAT THE NEXT SUBTASK CAN BE EXECUTED. ALL SUBTASKS END WITH LOCAL ACTIONS THAT REQUIRE THE NSC. AN ASTERISK IS USED FOR CONSECUTIVE PEELING SETUPS. EVERY SUBTASK IS TRAINED WITH A SEPARATE MODEL; SUBTASKS DO NOT SHARE MODEL PARAMETERS.

Index	Subtask	Goal	Network type	ATC	NSC	Index	Subtask	Goal	Network type	ATC	NSC
1	*GraspBanana*		Global	✗	✗	6	*ReachRight*		Global → Local	✓	✓
2	*PickUp*		Global	✗	✗	7	*PeelRight*		Local → Global	✓	✗
3	*GraspNeck*		Global → Local	✓	✗	8	*Reposition*		Global	✗	✗
4	*Push**		Hand-crafted	✗	✗	9	*ReachLeft*		Global → Local	✓	✓
5	*PeelNeck*		Local → Global	✓	✗	10	*PeelLeft*		Local → Global	✓	✗

Dataset	Number of demos	Total demo time (min)
GraspBanana	2050	89.9
PickUp	2073	63.2
GraspNeck	2141	80.9
PeelNeck	256	35.6
ReachRight	4458	211.6
PeelRight	264	27.4
Reposition	1779	41.3
ReachLeft	4009	237.6
PeelLeft	289	23.7
Total	17319	811.1

TABLE II
TRAINING SET STATISTICS.

the goal state in a modified form; the relative goal state $s_T^{local} = s_T - s_t = \pi_{local}^{goal}(o_t)$ is inferred and used to predict the reactive action, where π_{local}^{goal} is the goal predictor of the local-action network. Therefore, the reactive action is conditioned by the current foveated visual information and relative goal state: $a_t = s_{t+1} - s_t = \pi_{local}^{policy}(o_t, s_T^{local})$, where π_{local}^{policy} refers to the policy inference of the local-action network.

D. Model architecture

Figure 4 illustrates the model architecture.

1) Gaze predictor: The gaze predictor uses models similar to those described in [3]. The input 1280×720 raw image is resized into a 128×72 global image and then used to predict the gaze coordinate $x_{left}, y_{left}, x_{right}, y_{right}$ measured from the eye tracker while generating the demonstration data. Five subsequent convolutional layers and a SpatialSoftmax layer [1] process the global image to predict two-dimensional feature coordinates, which then passes through a mixture density network (MDN) [55], [56] to fit a Gaussian mixture model (GMM) with eight Gaussians into two-dimensional stereo gaze coordinates using the following loss:

$$\mathcal{L}_{gaze} = -log\Big(\sum_{i=1}^{N'} w_t^i \mathcal{N}(h_t; \mu_t^i, \mathbf{\Sigma}_t^i)\Big), \tag{1}$$

where μ_t, Σ_t, and w_t respectively indicate the mean, covariance matrix, and the weight of each Gaussian, and h_t indicates the measured human gaze during the demonstration. The MDN used two fully connected (FC) layers with rectified linear units (ReLU) between layers.

During the test, the robot chooses stereo gaze coordinates that maximize the probability of the GMM. The stereo gaze coordinates are used to crop (in stereo) the RGB foveated image ($320 \times 180 \times 6$) from the raw stereo image ($1280 \times 720 \times 6$).

2) Local-action network: Local action outputs reactive action using the foveated image. The input foveated image o_t is processed using residual connections [57] to extract the visual feature f_t using convolutional layers. From this feature, the relative goal states $s_t^{local} = s_T - s_t$ are predicted using the MLP.

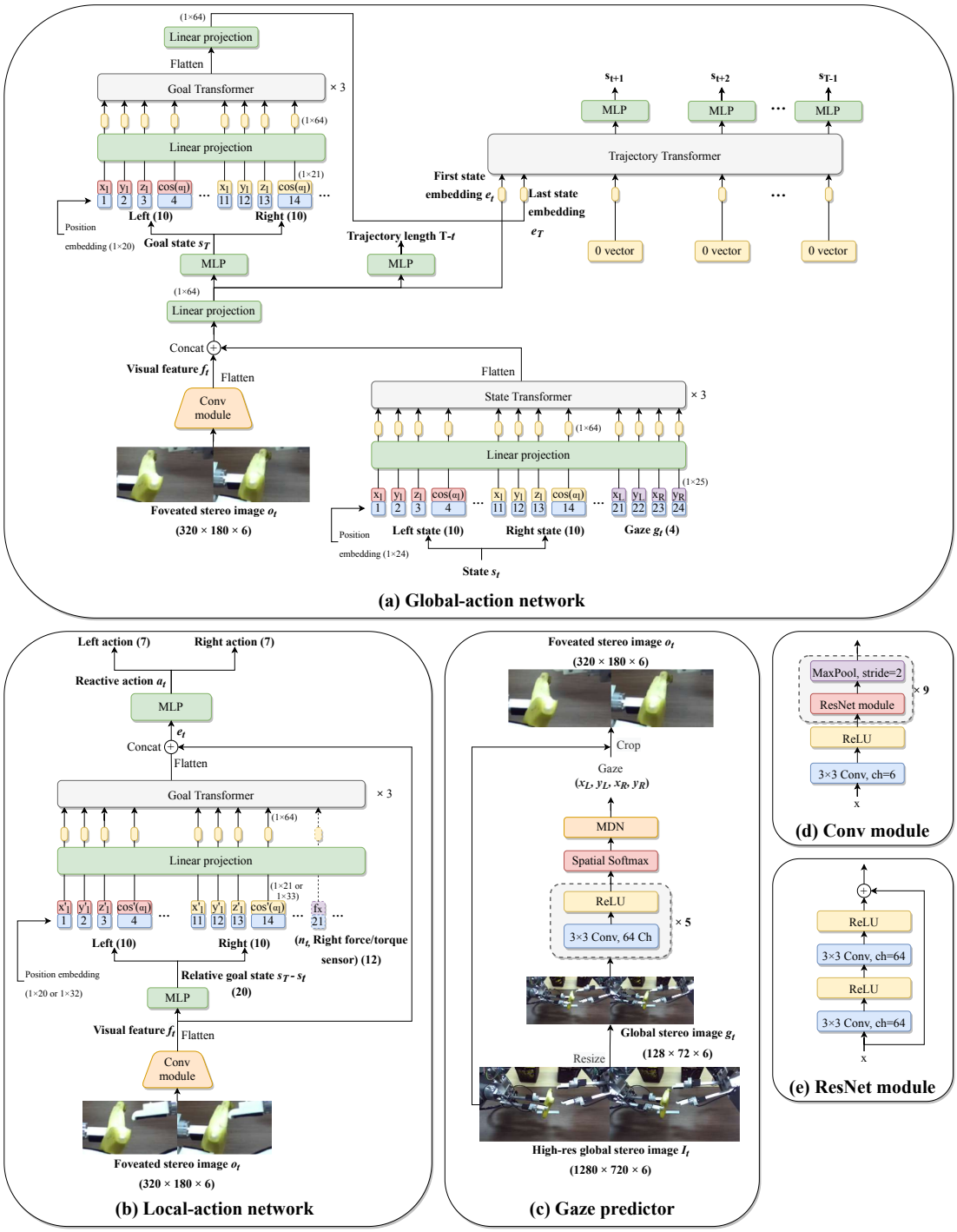


Fig. 4. Detailed neural network architecture of *GC-DA*.

Based on the results obtained from [4], which demonstrated that the Transformer-based self-attention mechanism can reduce distractions in high-dimensional somatosensory information, the relative goal state is processed using the Transformer encoder architecture (Goal Transformer) to focus on more important features. The output of the Goal Transformer is concatenated with f_t, and then processed by the MLP to output left/right reactive action. For *Peel∗* subtasks, as discussed in [52], the right hand's force/torque sensor value is also an input to the Goal Transformer so that the robot can also consider the tactile information during peeling.

3) Global-action network: The global-action network also inputs o_t to extract visual features f_t using the same ResNet-based architecture described in IV-D2. At this time, because the end-effector is usually out of the field of foveated vision, the left/right robot state and the gaze position are processed using the Transformer encoder (State Transformer) to minimize distraction. Here, e_t, which is a linearly projected concatenation of f_t and the State Transformer output, passes the MLP to predict the goal left/right state and

Algorithm 1 Goal-conditioned dual-action deep imitation learning

Parameters: Action-type classifier ATC, Global-action network π_{global}, Local-action network π_{local}, gaze predictor ρ

$t \leftarrow 0$ {initialize timestep}
while $\neg$ *succeed* **do**
  $I_t \leftarrow 1280 \times 720 \times 6$ raw stereo camera image
  $s_t \leftarrow$ left, right robot kinematics states
  $i_t \leftarrow \text{resize}(I_t, (128, 72))$ {resized stereo global image}
  $g_t \leftarrow \rho(i_t)$ {predicted gaze position}
  $o_t \leftarrow \text{crop}(I_t, g_t)$ {crop stereo foveated vision}
  if ATC(o_t) is *global* **then**
    $s_{t+1}, s_{t+2}, ..., s_T \leftarrow \pi_{global}(o_t, s_t, g_t)$
    $t' \leftarrow t$
    while ATC(o_t) is *global* **do**
      Execute $s_{t'}$ on robot
      $t' \leftarrow t' + 1$
    end while
    $t \leftarrow t'$
  else
    $a_t \leftarrow \pi_{local}(o_t)$ {local action}
    Execute a_t on robot
    $t \leftarrow t + 1$
  end if
  $succeed \leftarrow$ subtask success classifier {manual decision or automated (see Section V-F)}
end while

a length of trajectory $T - t$. Feature e_t is considered to be the initial embedding of the entire sequence, and the predicted goal state is processed by the Goal Transformer and linearly projected to create the last embedding. $T - t$ zero embeddings are generated to assign space for expected trajectory $s_{t+1}, s_{t+2}, ..., s_{T-1}$. All embeddings are gathered and processed by another Transformer encoder (Trajectory Transformer) to predict trajectory $s_{t+1}, s_{t+2}, ..., s_{T-1}$. The robot executes $s_{t+1}, s_{t+2}, ..., s_{T-1}$ and the inferred goal state s_T sequentially during $ATC(o_{t' \in \{t+1,...,T\}}) = global$.

In every neural network architecture, the Transformer that encodes the robot states, gaze, or force/torque sensory input uses a learned position embedding that passes a linear projection because its length is fixed. In addition, the Trajectory Transformer, which generates trajectory action, uses sinusoidal positional embedding [58] because it has to process the flexible length of the trajectory.

The MLP consists of three FC layers with ReLU between them and 200 nodes in each layer. The training minimizes the ℓ_2 loss between the model output and the ground-truth action. Other model training details are described in Appendix D. Every subtask described in IV-B is trained with a separate model; the subtasks do not share model parameters.

E. Global/local action separation

The proposed dual-action-based method is composed of global and local actions. As mentioned in III-B, this dual-action-based method requires separate training of the local action with the local-action network without disturbances from the global action. For example, in a demonstration sequence with timesteps $\{0, 1, 2, ..., T\}$, the steps between $[t_1, t_2]$ are annotated as local action steps, where t_1 and t_2 are determined from a criterion that decides whether to use local action. The entire set of timesteps $\{0, 1, 2, ..., T\}$ are considered to be the global action.

This subsection explains the criteria used to distinguish global/local actions for the *GraspNeck/Reach*$*$ and *Peel*$*$ subtasks, which are based on a basic criterion that selects reactive action when precise manipulation is required. Note that this action separation does not change the definition of the goal state; even after they are separated, the global and local actions share the same goal state s_T (e.g., in *GraspReach*$*$, goal states are the states where the end-effector reaches the target peel).

1) GraspNeck/Reach$*$ *subtasks:* A step is annotated as a local action step in the *GraspNeck/Reach*$*$ subtasks if the end-effector is in the foveated image (Fig. 17) because, when approaching the target, precise manipulation is required at the end of the sequence, which must perform the last-inch reaching to the exact location of the target [3], [54]. Because the *GraspNeck/Reach*$*$ subtasks contain many demonstrations (> 2140, Table II), a CNN-based classifier trained with a small amount of human-annotated data is used to annotate the entire dataset. The details of the automated annotation method can be found in Appendix C. The action-type classifier (ATC) is trained based on the annotated labels if the end-effector is in the foveated vision.

2) Peel$*$ *subtasks:* In *PeelNeck*, the criterion is whether the neck of the banana is peeled (Appendix B), because a force feedback-based high-precision reactive control is required to tear the neck without severely damaging the banana. After the peel is torn, no force feedback or highly precise reaching to the goal is required. All demonstrations were annotated manually, as the dataset is comparatively small (256 for training and 23 for validation). The annotated data are also used to train another ATC.

In *PeelRight* and *PeelLeft*, the criterion is relatively simple; a step is classified as a global-action step if the end-effector is closed. In these subtasks, the robot must control its end-effector precisely using reactive action during closing so that after complete closure, the end-effector holds the peel. By contrast, the peeling behavior after the holding requires the simple behavior of pulling the firmly held peel, which does not require high-precision manipulation.

V. Evaluation

A. Experiment setup

The banana-peeling task consists of nine subtasks from *GraspBanana* to *PeelLeft*, and each subtask requires the previous subtask to be successful. If all subtasks are conducted consecutively, we can test only a few samples on later subtasks such as *ReachLeft* or *PeelLeft*. Therefore, we tested each subtask by executing the subtask, then modifying the robot and banana position so that the robot could execute the next subtask. For example, if the robot dropped off the banana, it

TABLE III
ABLATION STUDY DETAILS.

	Local		Global		
Method	Action type	Goal conditioning	Action type	Goal conditioning	Vision type
(a) *GC-DA*	Reactive	✓	Trajectory	✓	Foveated
(b) *NoGC (Global)*	Reactive	✓	Trajectory	✗	Foveated
(c) *NoGC (Local)*	Reactive	✗	Trajectory	✓	Foveated
(d) *NoGC (Both)*	Reactive	✗	Trajectory	✗	Foveated
(e) *Reactive*	Reactive	✓	Reactive	✓	Foveated
(f) *Reactive-NoGC*	Reactive	✗	Reactive	✗	Foveated
(g) *Traj*	Trajectory	✓	Trajectory	✓	Foveated
(h) *Dual-resolution*	Reactive	✓	Trajectory	✓	Global

was placed in the robot's hand again. If the end-effector of the robot failed to reach the peel during *ReachRight*, a human manually re-adjusted the robot's end-effector so that it reached the peel. Therefore, even if the robot failed at the previous subtask, the next subtask trial could be evaluated under the same conditions as those used for the trials with successful previous subtasks.

To maintain the condition of the bananas, we bought bananas at the local market and used them on the same day. Each banana was cut to a depth of 1 cm because even the human operator could not peel the banana through teleoperation because its neck was too stiff to break. We found that the banana became peelable through teleoperation after it had ripened for 2 days. However, the banana also became soft and easily breakable, and controlling the banana's condition became difficult.

B. Ablation studies

This study evaluated the following questions:

- Is the proposed dual-action system valid?
- Is goal-conditioned action inference effective?

The ablation studies presented in Table III were designed to solve the above questions. Method (a) *GC-DA* fully utilizes the proposed dual-action architecture and goal-conditioned action inference. The *NoGC* methods ((b)–(d)) do not use the goal-conditioned action inference for global action (b), local action (c), or both (d). Method (e) *Reactive* examines the performance of reactive global action. Method (f) *Reactive-NoGC* combines methods (d) *NoGC (both)* and (e) *Reactive*; global/local actions are not conditioned by the goal state and are reactive. In *Reactive* and *Reactive-NoGC*, global actions are computed from foveated vision, robot state, and gaze. In contrast to method (e), in method (g) *Traj*, both global and local actions are trajectories. In this method, the local action adopts the network architecture of the trajectory-based global-action network, but only inputs the foveated vision. To evaluate the combination of global action with the global vision proposed in [3], method (h) *Dual-resolution* tested a trajectory-based global action inferred from the global image. In method (h), the global image was processed by five convolutional layers combined with a SpatialSoftmax layer [3], [5]. (i) *Diffusion policy* [59], a recently proposed method that generates visuomotor policy using a denoising diffusion process, is tested to determine whether multimodality residing in human-generated demonstration data can be addressed through the conditional denoising diffusion process. The network architecture employs the CNN-based model described in [59], utilizing the foveated stereo image, gaze, and left/right state as input. This is to maintain consistency with other tested architectures and to enhance robustness against visual distractions in the background. The same hyper-parameters as in [59] were used. We note that, since *Diffusion policy* generates an action trajectory, it is equivalent to the trajectory-based action rather than to reactive action.

C. Experiment result

Each ablation study method was tested with 15 bananas. Details of the evaluation criteria are described in Appendix E.

Table IV presents the experiment results. *PickUp* and *Reposition* were not evaluated because these subtasks are intermediate behaviors that connect the previous subtask to the next subtask; therefore, an evaluation metric would be ambiguous. The proposed *GC-DA* recorded the highest mean score. Remarkably, *GC-DA* scored the highest of all the *GraspNeck/Reach*$*$ subtasks. Some ablation methods without the goal-conditioning (methods (b) and (c)) recorded higher scores in *PeelRight* and *PeelLeft*, respectively, but all ablation methods without *GC* ((b)–(d)) recorded lower accuracy in the *GraspNeck/Reach*$*$ subtasks. This result infers that goal-conditioned action inference improves manipulation accuracy during precise reaching. By contrast, inferring the exact goal position is not essential for the *Peel*$*$ subtask.

When the global-action network output is reactive instead of a trajectory (methods (e)–(f)) and the local action predicted a trajectory (method (g)), performance dropped in the *GraspNeck/Reach*$*$ subtasks. These results also show that the combination of reactive local action and trajectory-based global action is essential.

The result for method (h) *Dual-resolution* indicates that the dual-resolution architecture, in which the global action is computed from the global vision, degrades the reaching performance. In our previous research, we adopted a dual-resolution architecture [3], and two-dimensional translation of end-effector was required for global action, which was sufficient with the low-resolution global vision. However, in the banana-peeling task, during global action, the robot must calculate a robust and exact three-dimensional trajectory to reach the vicinity of the banana peels, which requires high-

TABLE IV
RESULTS OF THE ABLATION STUDIES. G AND L REFER TO GLOBAL-ACTION AND LOCAL-ACTION NETWORK TYPES REPRESENTED IN TABLE I.

Method	*GraspBanana* G	*GraspNeck* G → L	*PeelNeck* L → G	*ReachRight* G → L	*PeelRight* L → G	*ReachLeft* G → L	*PeelLeft* L → G	Mean
(a) *GC-DA*	**1.000**	**0.933**	0.833	**0.933**	0.750	**0.846**	0.792	**0.870**
(b) *NoGC (Global)*	0.867	0.667	0.633	0.692	0.679	0.533	**0.833**	0.701
(c) *NoGC (Local)*	**1.000**	0.867	0.600	0.667	**0.893**	0.462	0.667	0.736
(d) *NoGC (Both)*	**1.000**	0.800	0.200	0.467	0.643	0.800	0.731	0.663
(e) *Reactive*	0.867	0.400	0.800	0.583	0.708	0.700	0.778	0.691
(f) *Reactive-NoGC*	**1.000**	0.600	**0.929**	0.600	0.667	0.692	0.792	0.754
(g) *Traj*	**1.000**	0.000	0.833	0.000	0.571	0.231	N/A	0.439
(h) *Dual-resolution*	**1.000**	0.600	0.700	0.500	0.775	0.714	0.607	0.696
(i) *Diffusion policy* [59]	0.889	0.111	N/A	N/A	N/A	N/A	N/A	N/A

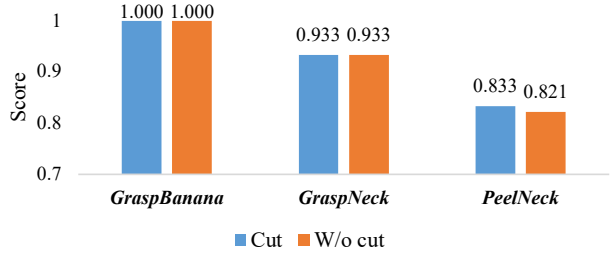
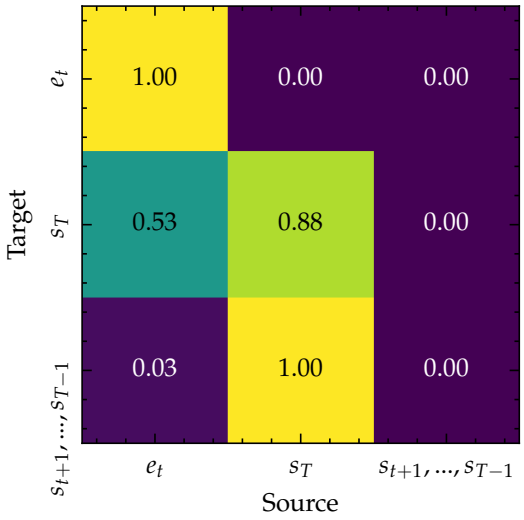
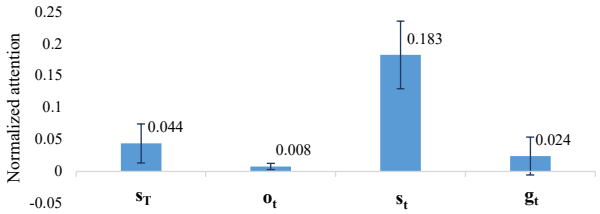


Fig. 5. Scores up to the *PeelNeck* subtask for experiments using the cut bananas (blue) and the two-day ripened bananas without the cut (orange). The result indicates that there is no difference between the two.



(a) Attention map of trajectory-based global action.



(b) Attention map of reactive global action.

Fig. 6. Attention maps of trajectory-based and reactive global actions. The trajectory $s_{t+1}, ..., s_{T-1}$ attended to goal state s_T (Fig. 6a), whereas reactive global action attended more to initial state s_t (6b).

resolution foveated vision for depth computation and the suppression of background distractions [5].

The results for (i) *Diffusion policy* indicate that it was successful for *GraspBanana* but failed with *GraspNeck*. This suggests that the *Diffusion policy* might not be suitable for tasks that demand high precision.

Comparing the outcomes of (a) *GC-DA*, (g) *Traj*, and (i) *Diffusion Policy*, it can be concluded that methods based on trajectory output are unsuitable for precise manipulation, thus necessitating reactive local action. These results underscore the importance of distinguishing between and learning trajectory-based global and reactive local actions separately.

Figure 5 presents the manipulation score up to the *PeelNeck* subtask for both cut bananas (Table IV (a)) and bananas ripened for 2 days without the cut. Both showed similar scores, indicating that the banana can be peeled without cutting with enough ripening.

D. Trajectory inference analysis

This subsection examines how trajectory-based global action outperforms reactive global action. The enhanced performance is attributed to the trajectory-based global action being more influenced by the goal state compared to its reactive counterpart, thereby resulting in a more stable policy. For all validation sets across all subtasks, attention maps were computed for both trajectory-based global action (Table IV (a)) and reactive global action (Fig. 6).

These attention maps were computed by averaging the attention rollout [60] of the Transformer's attention weights in embeddings s_t, s_T, and $[s_{t+1}, ..., s_{T-1}]$ individually. The result clearly shows that the generated trajectories (target: $[s_{t+1}, ..., s_{T-1}]$) are heavily conditioned on the inferred goal state (source: s_T, Fig 6a). By contrast, Fig. 6b indicates that the reactive global action is heavily conditioned on current robot kinematic state s_t and less on goal state s_T (see Appendix H for details of the attention computation).

E. Analysis of the goal conditioning

This subsection analyzes the typical necessity of goal conditioning during reach subtasks. Figure 7 visualizes the accuracy of goal state inference on the validation set. In this figure, the mean Euclidean distance between the ground-truth goal

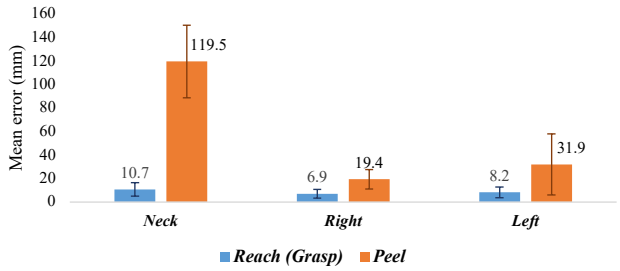


Fig. 7. Error comparisons of goal state prediction in reach and peel subtasks obtained by global-action networks.

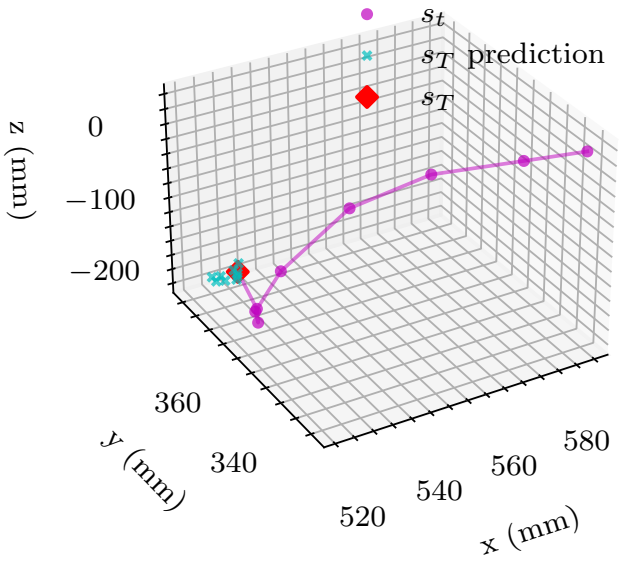


Fig. 8. Example recorded trajectory and the predicted goal states in *GraspNeck* visualized. State s_T prediction indicates the predicted goal state s_T from every state s_t.

state and the predicted goal state of the right arm computed by the global-action network for every timestep is shown. The Euclidean error is low (< 11 mm) for reach subtasks (blue) but high for peeling subtasks (orange). This result indicates that peeling subtasks are less conditioned by goal state; i.e., peeling can be ended anywhere after the peel is torn, but reaching needs to be ended at the exact location at the banana's peel or neck. The above result supports the result in Table IV (b) and (c), where non-goal-conditioned (*NoGC*) models sometimes showed higher peeling scores than *GC-DA*, as peeling subtasks require less goal conditioned action inference. Moreover, even though a goal prediction accuracy of < 11 mm is enough for many pick-and-place tasks, it is not sufficient for reaching the peel in our task setup; therefore, recursive last-inch reaching is required.

Figure 8 visualizes a recorded trajectory $s_0, ..., s_T$ in an episode in the validation set of *GraspNeck* and the predicted goal state s_T from each state s_t, $t \in [0, T)$. The goal state can be predicted regardless of current state s_t.

F. Consecutive banana peeling

In previous sections, *GC-DA* was tested on each subtask respectively. However, in a real-world scenario, the robot must peel the entire banana with all subtasks in order, without human intervention. One problem in this scenario is that the final success rate may be dramatically low because failure accumulates throughout the subtasks; the success of the previous subtask is usually required for the success of the current subtask. Another problem is that the initial posture and states of the banana cannot be manually adjusted at the start of each subtask because no human intervention is permitted. Therefore, the robot may have to start a subtask even though the banana is in a state in which the robot cannot accomplish the subtask. To solve these problems, it is necessary to (1) maintain a success rate that is as high as possible for each subtask and (2) maintain a stable grasping posture of the banana while peeling. To this end, we implemented a few modifications.

1) Modifications for consecutive banana peeling: First, the gripper of the left arm was re-designed to be concave so that the robot can grasp the banana more stably. This modification reduces slipping of the banana while it is being peeled (see the left end-effector in Fig. 10).

Second, a pre-defined behavior that pushes the banana's neck 30 mm to the left between *GraspNeck* and *PeelNeck* was adopted to prevent the banana from tilting. Because of the simplicity of this behavior, the pushing action was manually designed.

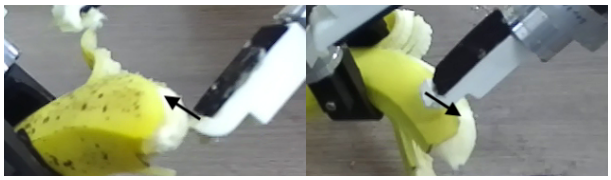
Third, we found that additional reaching data from hard-to-reach cases were helpful. Figure 9 illustrates examples of such cases; for example, Fig. 9d illustrates an additional demonstration data sample that assumes that the end-effector missed the target peel and is placed on the left side of the peel during *ReachLeft*. The end-effector is required to move back to the peel using a policy that is not usually acquired during a normal demonstration, so an intentionally generated dataset of such state helps the robot to handle failure cases. For subtasks *ReachLeft* and *ReachRight*, we defined two hard-to-reach cases (*FromRight* and *FromLeft* in each subtask (Fig. 9), then generated the demonstrations.

These demonstrations were used in two ways. First, they were integrated into the dataset of Table II to increase the number of total demonstrations used for training. Second, each case was separately trained with another local-action network that takes charge when the robot faces each hard-to-reach case. A simple CNN-based classifier is trained with the foveated images to classify which case the current state is in: the two hard-to-reach cases (*FromLeft* or *FromRight*), or the normal cases. The training dataset statistics can be found in Table V. During the test, the classifier determines which local-action network to use. Finally, we added additional demonstration sets as well as the demonstrations for the hard-to-reach cases. These demonstrations were used to train the model with the demonstration sets in Table II. Training dataset statistics for the additional demonstrations can be found in Table V.

2) Transition between subtasks: There are two different types of transitions between adjacent subtasks. First, when the earlier subtask ends with trajectory inference (*GraspBanana*,



(a) *ReachRight-FromRight*. (b) *ReachRight-FromLeft*.



(c) *ReachLeft-FromRight*. (d) *ReachLeft-FromLeft*.

Fig. 9. Hard-to-reach cases for *ReachRight* and *ReachLeft*.

Dataset	Number of demos	Total demo time (min)
GraspBanana	1429	76.7
PickUp	868	29.0
GraspNeck	1359	55.5
PeelNeck	149	18.5
ReachRight (total)	4233	184.6
(normal)	2029	90.3
FromRight	1174	51.1
FromLeft	1030	40.2
PeelRight	156	21.3
Reposition	158	4.37
ReachLeft (total)	3355	159.8
(normal)	1284	74.4
FromRight	1125	52.0
FromLeft	946	33.3
PeelLeft	146	12.3
Total	11853	561.8

TABLE V
ADDITIONAL TRAINING SET STATISTICS.

PickUp, and *Reposition*), and the *Peel*$*$ subtasks, later subtasks can be conducted as soon as the current trajectory execution is done. Otherwise, if the subtask ends with a reactive action (*GraspNeck/Reach*$*$ subtasks), the transition between subtasks requires additional classifiers.

The robot changes *GraspNeck* to *PeelNeck* when the right end-effector is closed. *ReachLeft* and *ReachRight* use additional CNN-based classifiers (the NSC). In *ReachRight*, the NSC is trained with classified foveated images in *ReachRight* and *PeelRight*; if the current foveated image o_t is classified as *PeelRight*, the next subtask for execution is changed to *PeelRight*. In *ReachLeft*, another NSC classifies foveated images in *ReachLeft* and *PeelLeft*. The NSC comprises the convolution module described in Fig. 4 (d) and a three-layer MLP.

3) Result: Figure 10 illustrates an example of the successful behavior of *GC-DA* in the consecutive banana peeling setup. By contrast, Fig. 11 shows failure examples for the same setup, including examples in which the robot failed to reach the end-effector to the target (Fig. 11a) or the neck was broken during peeling (Fig. 11b).

Figure 12 presents the score of consecutive banana peeling with 22 bananas. *Independent* considers only the executed trials for each subtask: *Independent* $= \sum_i^N score_i/N$ for $N <= 22$ is the number of trials actually executed in each subtask. For example, if $21/22$ trials succeeded to perform *GraspNeck*, the N for *PeelNeck* is 21. *Dependent* also considers unexecuted trials: *Dependent* $= \sum_i^{22} score_i/22$. Note that the *Dependent* score for *ReachLeft* can be higher than the *Dependent* score for *PeelRight* because the robot can attempt the *ReachLeft* subtask even though it has failed at the *PeelRight* subtask.

TABLE VI
SUCCESS RATE FOR EACH PEELING TASK AND TOTAL SUCCESS RATE

Subtask	*PeelNeck*	*PeelRight*	*PeelLeft*	*Total*
Success rate (%)	95.45	59.09	63.63	40.91

TABLE VII
TASK SUCCESS RATE COMPARISON OF *GC-DA*, *Reactive-NoGC*, AND *Diffusion policy* ON *MoveBowl* AND *PickCoin*.

Task	*GC-DA*	*Reactive-NoGC*	*Diffusion policy*
MoveBowl	**93.3%**	0.0%	6.7%
PickCoin	**70.0%**	30.0%	5.0%

Table VI reports each peeling subtask's success rate and the overall subtask success rate. The success rate is calculated for trials with *scores* > 0.5.

G. Applicability of GC-DA

To demonstrate the applicability of the proposed method to other manipulation tasks, we trained and tested the model on two additional tasks: *MoveBowl* and *PickCoin*. As illustrated in Fig. 13, in *MoveBowl*, the robot supports the bowl from the right and left sides with both arms and move the bowl to the tray. This task requires dual-arm manipulation and balancing of the bowl while moving. It is divided into three subtasks: moving the right end-effector to the right side of the bowl (*SupportRight*), moving the left end-effector to the left side of the bowl (*SupportLeft*), and moving the bowl to the tray with both arms (*Move*). In *PickCoin*, the robot picks up a thin coin from the table, which requires precise control. Table VIII presents the training set statistics for both tasks.

Table VII presents the results for both tasks. In each case, the final success rate was significantly enhanced by using the proposed *GC-DA*. *Diffusion policy* reached the vicinity of the target object but failed at precise object manipulation (e.g., picking up a coin, or accurately supporting the bowl with both arms), indicating the necessity of reactive local action for dexterous, high-precision manipulation. Additionally, to

TABLE VIII
TRAINING SET STATISTICS OF *PickCoin* AND *MoveBowl*. G AND L REFER TO GLOBAL-ACTION AND LOCAL-ACTION NETWORK TYPES.

Dataset	Number of demos	Total demo time (min)
PickCoin (G $\rightarrow$ *L*)	3051	168.7
MoveBowl (total)	2430	84.7
SupportRight (G $\rightarrow$ L)	840	24.4
SupportLeft (G $\rightarrow$ L)	712	30.4
Move (G)	878	29.9

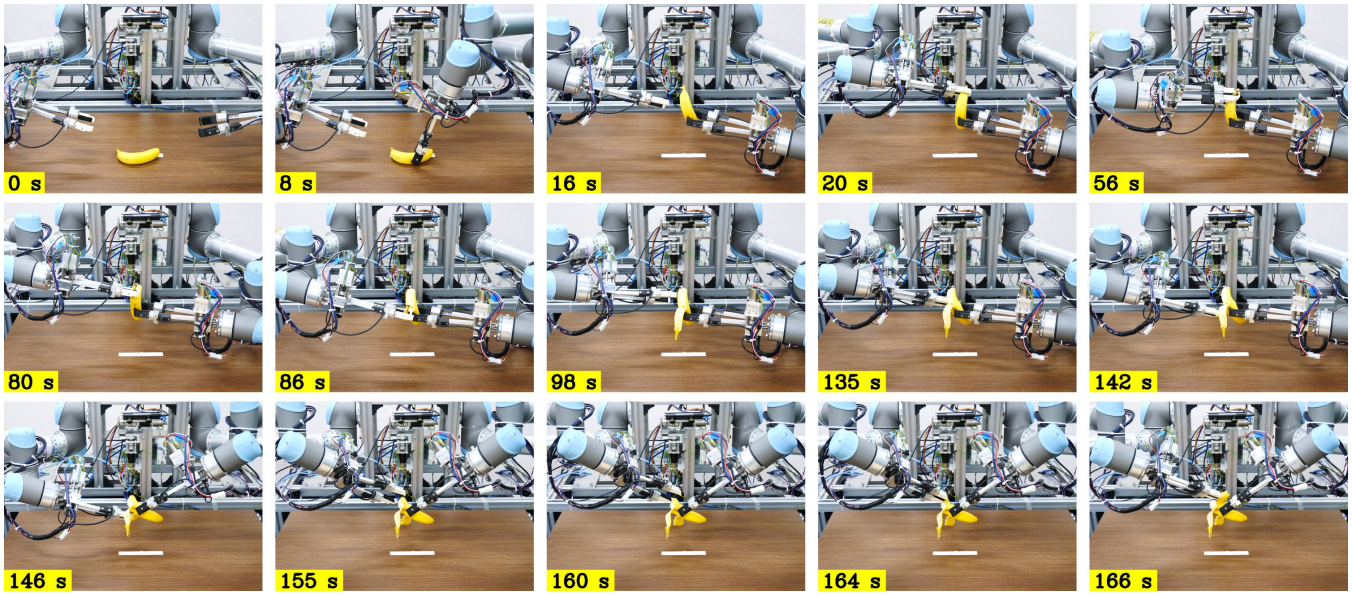
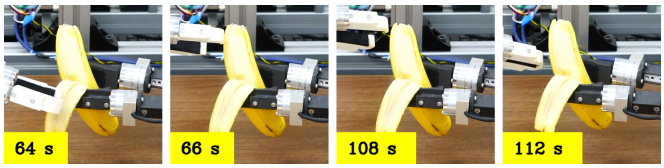
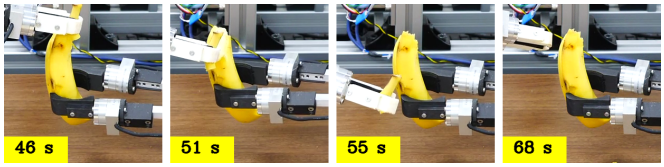


Fig. 10. Example of a successful trial. The robot grasped and picked up the banana (0–16 s), reached its right end-effector to the neck (16–20 s), then peeled the neck (20–86 s). Then, the robot reached its right end-effector to the right side of the banana and peeled it (86–142 s). Finally, the robot rotated the banana (146 s) and peeled the left part (146–166 s).



(a) The robot failed to reach its end-effector to the peel.



(b) The banana's neck was broken during peeling.

Fig. 11. Failure examples of banana peeling.

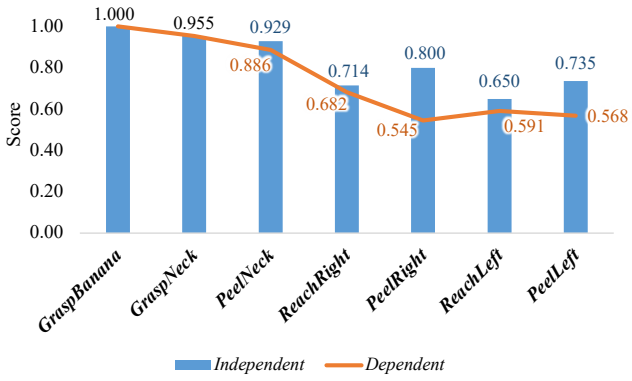
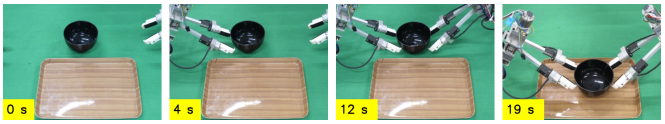
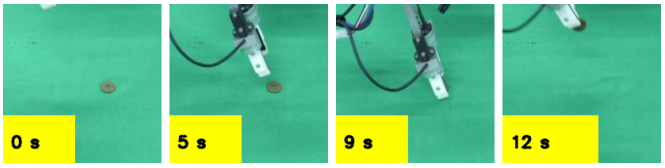


Fig. 12. Score of each subtask in consecutive banana peeling. *Independent* refers to the score of each subtask; therefore it does not consider failures from previous subtasks, whereas *Dependent* also counts failures from the previous subtask.



(a) *MoveBowl*: robot supports right and left side of the bowl with its both arms and transport it to the tray.



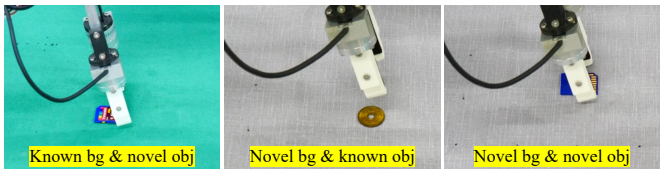
(b) *PickCoin*: robot picks coin on the table.

Fig. 13. Example successful trials of *MoveBowl* and *PickCoin* with *GC-DA*.

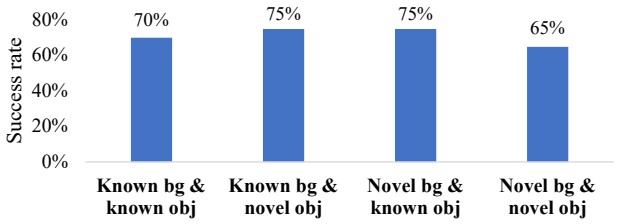
demonstrate the learned policy's robustness to novel objects or backgrounds, further experiments were conducted with *PickCoin* using a novel object (a memory card) and/or a novel background (gray) (Fig. 14a). The results indicate that the proposed model can successfully adapt to a novel object and/or background (Fig. 14b).

H. Robustness of gaze prediction

The robustness of gaze prediction against visual distractors in the background is crucial, as the robot may need to execute object manipulation in novel environments. We evaluated the gaze predictor using a teleoperated banana peeling dataset, incrementally increasing the number of distractors in the background. Figure 15a presents the mean Euclidean error in pixels with various numbers of distractors, showing an increasing tendency of the error as the number of distractors rises. A

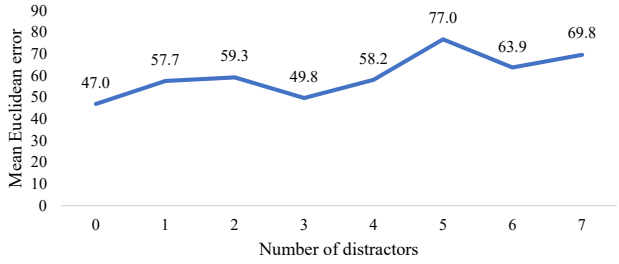


(a) Configurations with varying combinations of known and novel objects and backgrounds.

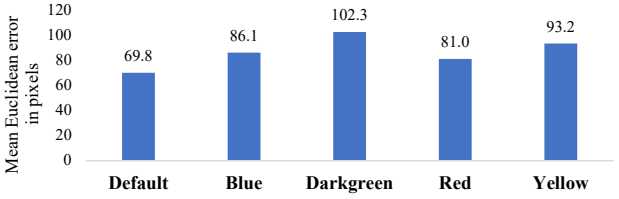


(b) Success rates.

Fig. 14. Robustness of *GC-DA* with various combinations of known and novel objects (obj) and backgrounds (bg).



(a) Error by number of visual distractors.



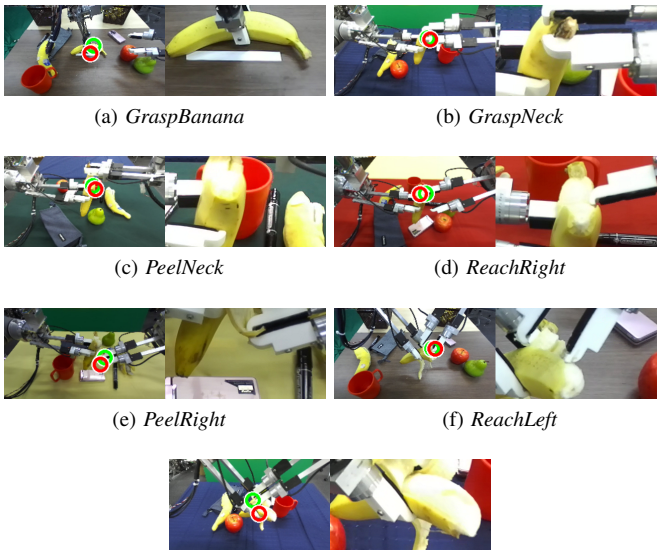
(b) Error across different backgrounds with seven distractors.

Fig. 15. Mean Euclidean distance error in pixels of the gaze prediction against the number of visual distractors in the background and background variability.

slight increase in the error was also observed with various backgrounds (Fig. 15b). However, as depicted in Fig. 16, even in the most complicated scene (with 7 distractors) and various backgrounds, the predicted gaze successfully attends to the target banana, effectively excluding background distractors.

VI. Conclusion

This study realized the banana-peeling task with a general-purpose dual-arm robot (UR5) based on the proposed (*GC-DA*) deep imitation learning. The proposed method separates the policy of robot motion into policies for global and local actions. A trajectory-based global action was used to deliver the end-effector stably close to the goal position when the robot does not require very precise manipulation of the target object, such as when delivering the robot arm to the vicinity



(a) *GraspBanana* (b) *GraspNeck*

(c) *PeelNeck* (d) *ReachRight*

(e) *PeelRight* (f) *ReachLeft*

(g) *PeelLeft*

Fig. 16. Example gaze prediction results in a cluttered environment (7 distractors with various backgrounds). The left images display the human gaze (green) and predicted gaze (red) on the left camera image, while the right images showcase the foveated vision generated using the predicted gaze. Even with a background cluttered with various objects, the predicted gazes successfully focused on the target while excluding distractors.

to the banana peel. The reactive local action precisely controls the end-effector to manipulate the target object, such as during the precise last-inch reaching to the banana peel. The inferred goal state conditions the model's output action for stable policy inference.

The proposed *GC-DA* method successfully executed the peeling of real bananas utilizing a dual-arm manipulator. The ablation study, as depicted in Table III, reveals that the reactive local action was indispensable for achieving manipulation skills with high precision, while the trajectory-based global action proved essential for tasks with a long horizon. Consequently, for tasks requiring precise manipulation over long horizons, a combination of both trajectory-based global action and reactive local action was necessary.

This result is important for two reasons: first, highly dexterous, long-horizon robot manipulation of a deformable object is possible without a task-specific gripper. Second, the above dexterous manipulation skills were acquired from human-generated behavior data. To this end, the proposed method accomplished the banana-peeling task, which is a combination of many different sub-tasks for grasping the banana, precisely grasping each peel, then peeling, which requires dexterity.

In this research, $811.1 + 561.8$ minutes ≈ 21.8 hours of demonstration data were generated and used for training, which can be obtained in only a few days. Considering the complexity of the target banana-peeling task, we believe that 21.8 hours of demonstration is feasible. However, data efficiency has always been pursued in learning-based robotics. Therefore, our future interest is to study whether few-shot learning of highly dexterous manipulation skills is possible by transferring knowledge about objects and behaviors. Previously, the importance of behavior segmentation and structured

TABLE IX
ACCURACY OF THE ATC ON THE TRAINING AND VALIDATION SETS.

Subtask	Training (%)	Validation (%)
GraspNeck	94.91	96.48
PeelNeck	94.23	88.24
ReachRight	93.69	93.33
ReachLeft	93.29	93.51

TABLE X
ACCURACY OF THE NSC ON THE TRAINING AND VALIDATION SETS.

Subtask	Training (%)	Validation (%)
ReachRight	97.59	94.99
ReachLeft	98.19	97.52

TABLE XI
NUMBER OF ANNOTATED, CLASSIFIED BY THE TRAINED MODEL, AND RE-ANNOTATED DEMONSTRATIONS ON EACH SUBTASK.

Subtask	# of annotated	# of classified	# of re-annotated
GraspNeck	283	2087	369
ReachRight	400	4546	288
ReachLeft	400	4065	202

behavior and tasks have been reported [25], [61], [62]. Because unsupervised segmentation is possible with gaze [63], and gaze information is highly correlated with task objective [64], [65], we hypothesize that gaze can be applied for the unsupervised segmentation and structuring of tasks and further be applied to the transfer of behavior to other tasks. The application of gaze-based human behavior segmentation for structured deep imitation learning is a promising topic for future research.

This research demonstrated *GC-DA* with the banana-peeling task. Even though the proposed method still requires a few human engineering factors, such as manual task segmentation and dual-action annotation, the proposed method greatly diminishes the amount of manual engineering needed to develop the dexterous manipulation skills required for banana-peeling tasks. Also, our approach relies on a natural semantic segmentation strategy (e.g., "grasp the neck of the banana," "peel the neck of the banana") align well with the semantic structure of the task. Each subtask is defined as a combination of local and global actions which can be easily defined based on the required dexterity (Fig. 3). In this context, we believe that if users wish to apply our learning-based approach to their own applications, they can perform segmentation in a human-natural manner and allocate local actions according to the necessary dexterity for challenging tasks, such as precise reaching or object manipulation. Therefore, the *GC-DA* schema can also be used for other manipulation tasks that require dexterous skills that cannot be defined by human-engineered rules. For example, our approach is a promising method for various other bimanual manipulation tasks, such as cutting or peeling vegetables (e.g., green onions or onions), where one hand needs to hold the object and the other hand performs the task, and precise location and orientation of the end-effectors are crucial. In such cases, the proposed dual-action can effectively succeed in approaching the end-effector (i.e., knife) to near the target place and actually cut the onion with the reactive local-action. Another potential application of the proposed method is factory automation of deformable objects, such as smartphone box packaging, cable manipulation, or food manipulation. We expect further research will apply the *GC-DA* to those manipulation tasks while reducing the remaining effort needed for manual annotation.

APPENDIX

A. Performance of ATC and NSC

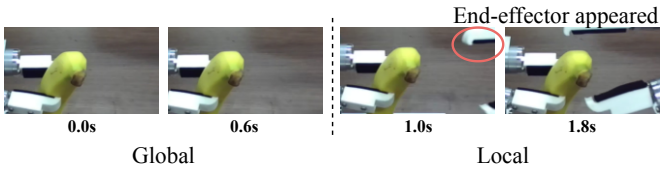
This subsection presents the accuracy results of the ATC and NSC on the training and validation sets (Tables IX and X).

B. Boundaries between global/local actions in the GraspNeck/Reach∗ subtasks

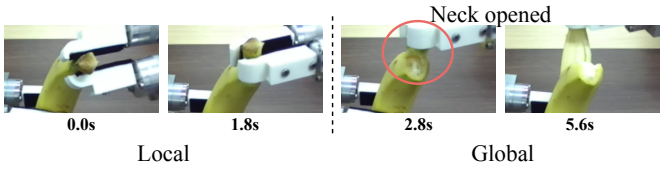
Figure 17 illustrates examples of boundaries between global and local actions in each subtask. In the *GraspNeck/Reach∗* subtasks, a step is identified as a local action if the effective end-effector is in the foveated image (Figs. 17a, 17c, and 17e). In *PeelNeck* (Fig. 17b), a behavior that opens the banana's neck is identified as a local action, and the peeling behavior is identified as the global action. In *PeelLeft* (Fig. 17d) and *PeelRight* (Fig. 17f), grasping the banana peel is defined as a local action, and peeling behavior is identified as a global action.

C. Annotation method

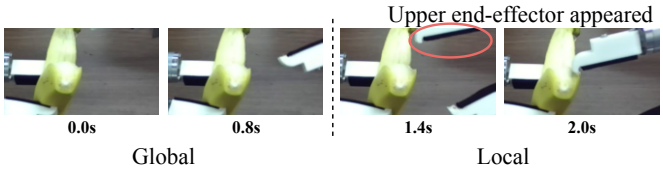
To reduce the time required for annotation, we used an image classifier for automated annotation similar to [66]'s. First, a human manually annotated part of the entire episode into global/local actions. For example, a human annotated 283 out of 2370 episodes in the *GraspNeck* subtask. Then, a simple CNN-based image classifier was trained using the annotated dataset (90% training, 10% validation). This CNN-based classifier comprised ten consecutive convolutional layers with ReLU, and a maxpooling layer was placed between each convolutional layer. The output of the convolutional layers was passed through two layers of MLPs, which returned the following binary classification results: $C(o_t) \rightarrow \{\textit{global action}, \textit{local action}\}$, where C indicates the trained classifier. Next, the trained classifier annotated the unannotated episodes. To filter false classification results, we utilized the fact that the transition between global and local actions occurs only once. In the *GraspNeck/Reach∗* subtasks, if $C(o_t) \rightarrow$ *local action* and $C(o_{t+1}) \rightarrow$ *global action* for any t in timestep $0, ..., T-1$ in an episode of length T, this episode was re-annotated by the human. For the *Peel∗* subtasks, the human re-annotated the episodes with $C(o_t) \rightarrow$ *global action* and $C(o_{t+1}) \rightarrow$ *local action*. Table XI presents the number of demonstrations annotated by humans that were used for the model training, classified demonstrations by the trained model, and re-annotated demonstrations.



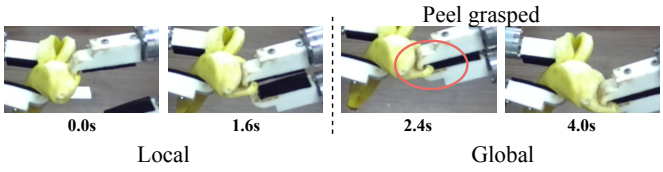
(a) *GraspNeck*.



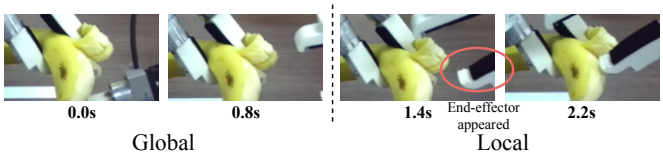
(b) *PeelNeck*.



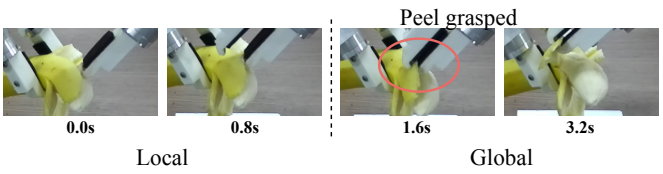
(c) *ReachRight*. The effective end-effector is the upper end-effector.



(d) *PeelRight*.



(e) *ReachLeft*. The effective end-effector is the lower end-effector.



(f) *PeelLeft*.

Fig. 17. Boundaries displayed in the foveated image.

D. Model training details

This research followed the model training details described in [52]. A rectified Adam optimizer [67] with a learning rate of $1e-5$ for the global and local-action networks and $1e-4$ for the gaze predictors, a weight decay of 0.01, and a batch size of 16 were used to train the proposed model. Two AMP EPYC 7402 CPUs with four NVIDIA a100 and Intel Xeon CPU E5-2609 v4 with eight NVIDIA v100 GPUs were used for training the model. Because the *GraspNeck/Reach*$*$ subtasks are more complex than the other subtasks, they were trained for 500 epochs, and the other subtasks were trained for 300 epochs. During the execution on the robot, an Intel CPU Core i7-8700K and one NVIDIA GeForce GTX 1080 Ti were used.

E. Scoring criteria

The model was given a score of $+1$ if it achieved the objective of each subtask and $+0$ if it did not. In addition, on the *Peel*$*$ subtasks, specific scoring criteria were set. Detailed explanations of the criteria are as follows:

- *GraspBanana*: $+1$ if the robot grasps the banana without any harm. $+0$ if the robot destroys the banana or collides with the table.
- *GraspNeck*: $+1$ if the robot grasps the banana's neck. $+0$ if the robot fails to grasp the neck.
- *PeelNeck*: $+0.5$ for opening the neck and $+1$ for entirely peeling the neck.
- *ReachRight*: $+1$ if the upper end-effector touches the border between the right peel and pulp. $+0$ otherwise.
- *PeelRight*: If the bottom side of the peel is still attached, the robot scores $+0.5$. If it is entirely torn, the robot scores $+1$.
- *ReachLeft*: $+1$ if the lower end-effector touches the border of the left peel and pulp. $+0$ otherwise.
- *PeelLeft*: If the bottom side of the peel is still attached, the robot scores $+0.5$. If it is entirely torn, the robot scores $+1$.

Figure 18 visualizes examples of these criteria.

F. Translational and rotational uncertainties in foveated vision

Figure 19 illustrates a few samples of the initial step of the robot's perspective on subtasks *GraspNeck*, *ReachRight*, and *ReachLeft*. The translational uncertainties of the target object were efficiently managed using gaze, while rotational uncertainties remain in the foveated vision.

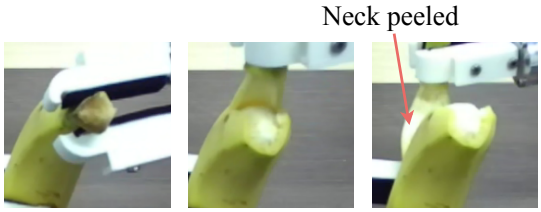
G. Data augmentation

Image augmentation is often used in computer vision to increase the robustness of the neural network to input data. This research randomly adjusted the brightness, contrast, saturation, and hue to transform the training input images. Additionally, random erasing [68] was applied to prevent overfitting of the neural network model to the specific features that only exist in the training data.

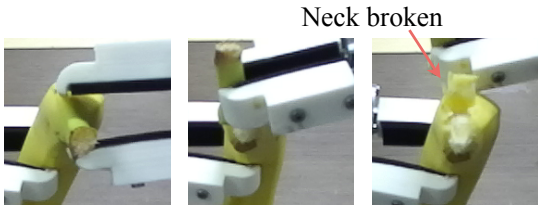
H. Attention computation

The computation of the attention map followed [66]. The trajectory-based global-action network inputs current state embedding e_t, inferred goal state s_T, and zero vectors $s_{t+1}, ..., s_{T-1}$. On this input sequence, attention rollout [60] $A^{\{T-t+1\}\times\{T-t+1\}}$ defined by the recursive multiplication of the attention weight of each layer in the trajectory Transformer is computed. To normalize the attention rollout on each target embedding, the attention rollout is divided by the maximum of target embedding values as follows:

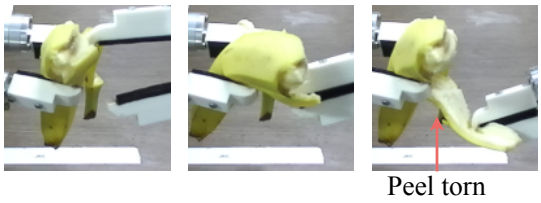
$$M_{i,j} = \frac{A_{i,j}}{max(A_i)}, \tag{2}$$



(a) *PeelNeck*: score = 1.



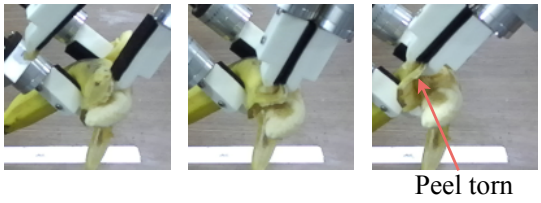
(b) *PeelNeck*: score = 0.5.



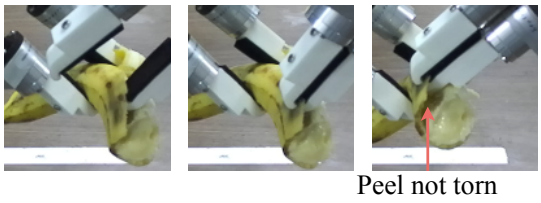
(c) *PeelRight*: score = 1.



(d) *PeelRight*: score = 0.5.



(e) *PeelLeft*: score = 1.



(f) *PeelLeft*: score = 0.5.

Fig. 18. Examples of scoring criteria.

where $i \in [0, T)$, $j \in [0, T)$ and M is the sequential attention map. Then, the attention on zero vectors $s_{t+1}, ..., s_{T-1}$ is averaged. The output attention map size is 3×3, where the elements are denoted as $[e_t, s_T, \{s_t + 1, ...s_T - 1\}]$.

Similarly, Eq. 2 normalizes an attention rollout of the reactive global-action network output. The Transformer output embeddings of the reactive global-action network are flattened and passed to an MLP to compute a_t. Therefore, the information about which source embedding is paid attention is more important than the attention map that considers the target embedding. The summation of target embedding corresponding to o_t, s_t, g_t, and s_T is computed by the following:

$$W^{s_T} = \sum_{i=0}^{138} \sum_{j=0}^{19} A_{ij}, \tag{3}$$

$$W^{o_t} = \sum_{i=0}^{138} \sum_{j=20}^{114} A_{ij}, \tag{4}$$

$$W^{s_t} = \sum_{i=0}^{138} \sum_{j=115}^{134} A_{ij}, \tag{5}$$

$$W^{g_t} = \sum_{i=0}^{138} \sum_{j=135}^{138} A_{ij}. \tag{6}$$

ACKNOWLEDGMENTS

This paper is partly based on results obtained under a Grant-in-Aid for Scientific Research (A) JP22H00528 and supported in part by the Department of Social Cooperation Program "Intelligent Mobility Society Design," funded by Toyota Central R&D Labs., Inc., of the Next Generation AI Research Center, The University of Tokyo.

REFERENCES

[1] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel, "Deep spatial autoencoders for visuomotor learning," in *International Conference on Robotics and Automation*, 2016, pp. 512–519.

[2] T. Zhang, Z. McCarthy, O. Jow, D. Lee, X. Chen, K. Goldberg, and P. Abbeel, "Deep imitation learning for complex manipulation tasks from virtual reality teleoperation," in *International Conference on Robotics and Automation*, 2018, pp. 1–8.

[3] H. Kim, Y. Ohmura, and Y. Kuniyoshi, "Gaze-based dual resolution deep imitation learning for high-precision dexterous robot manipulation," *Robotics and Automation Letters*, vol. 6, no. 2, pp. 1630–1637, 2021.

[4] H. Kim, Y. Ohmura, and Y. Kuniyoshi, "Transformer-based deep imitation learning for dual-arm robot manipulation," in *International Conference on Intelligent Robots and Systems*, 2021.

[5] ——, "Using human gaze to improve robustness against irrelevant objects in robot manipulation tasks," *Robotics and Automation Letters*, vol. 5, no. 3, pp. 4415–4422, 2020.

[6] D. Koert, G. Maeda, R. Lioutikov, G. Neumann, and J. Peters, "Demonstration based trajectory optimization for generalizable robot motions," in *International Conference on Humanoid Robots*, 2016, pp. 515–522.

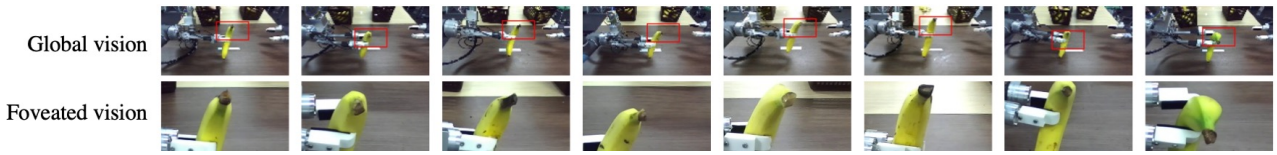
[7] A. Vakanski, I. Mantegh, A. Irish, and F. Janabi-Sharifi, "Trajectory learning for robot programming by demonstration using hidden markov model and dynamic time warping," *Transactions on Systems, Man, and Cybernetics*, vol. 42, no. 4, pp. 1039–1052, 2012.

[8] H. Kim, M. Yamada, K. Miyoshi, T. Iwata, and H. Yamakawa, "Reinforcement learning in latent action sequence space," in *International Conference on Intelligent Robots and Systems*, 2020, pp. 5497–5503.

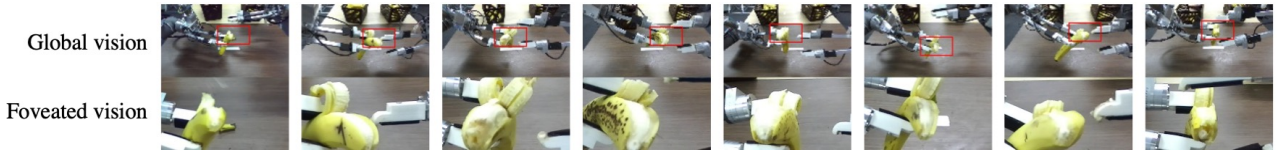
[9] M. Yamada, H. Kim, K. Miyoshi, T. Iwata, and H. Yamakawa, "Disentangled representations for sequence data using information bottleneck principle," in *Asian Conference on Machine Learning*, 2020, pp. 305–320.

[10] M. Turpin, K. Mohta, N. Michael, and V. Kumar, "Goal assignment and trajectory planning for large teams of interchangeable robots," *Autonomous Robots*, vol. 37, no. 4, pp. 401–415, 2014.

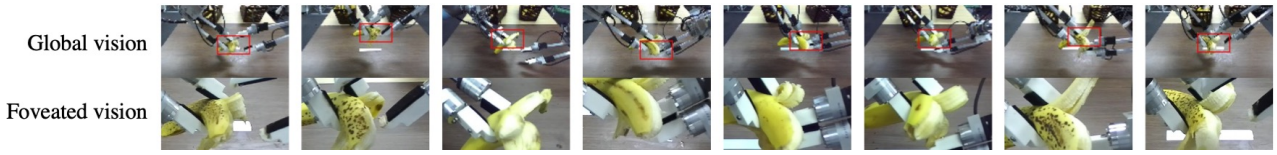
[11] S. Bhattacharya, R. Ghrist, and V. Kumar, "Persistent homology for path planning in uncertain environments," *IEEE Transactions on Robotics*, vol. 31, no. 3, pp. 578–590, 2015.



(a) *GraspNeck*.



(b) *ReachRight*.



(c) *ReachLeft*.

Fig. 19. Samples of the robot's view of subtasks *GraspNeck* (a), *ReachRight* (b), and *ReachLeft* (c). The robot effectively addresses the translational uncertainties of the target object by utilizing gaze, i.e., despite the target object potentially varying in global vision, gaze can effectively capture the target's position so that it always appears in the foveated vision.

[12] W. Van Loock, G. Pipeleers, M. Diehl, J. De Schutter, and J. Swevers, "Optimal path following for differentially flat robotic systems through a geometric problem formulation," *IEEE Transactions on Robotics*, vol. 30, no. 4, pp. 980–985, 2014.

[13] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (bit*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *International Conference on Robotics and Automation*, 2015, pp. 3067–3074.

[14] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Autonomous robot vehicles*, 1986, pp. 396–404.

[15] Y. K. Hwang, N. Ahuja, *et al.*, "A potential field approach to path planning." *IEEE Transactions on Robotics and Automation*, vol. 8, no. 1, pp. 23–32, 1992.

[16] K. H. Sedighi, K. Ashenayi, T. W. Manikas, R. L. Wainwright, and H.-M. Tai, "Autonomous local path planning for a mobile robot using a genetic algorithm," in *Congress on Evolutionary Computation*, vol. 2, 2004, pp. 1338–1345.

[17] K. Cole and A. M. Wickenheiser, "Reactive trajectory generation in an unknown environment," in *International Conference on Intelligent Robots and Systems*, 2017, pp. 2151–2157.

[18] R. A. Knepper, S. S. Srinivasa, and M. T. Mason, "Hierarchical planning architectures for mobile manipulation tasks in indoor environments," in *International Conference on Robotics and Automation*, 2010, pp. 1985–1990.

[19] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. P. Abbeel, and W. Zaremba, "Hindsight experience replay," in *Advances in Neural Information Processing Systems*, 2017, pp. 5048–5058.

[20] Y. Ding, C. Florensa, P. Abbeel, and M. Phielipp, "Goal-conditioned imitation learning," in *Advances in neural information processing systems*, vol. 32, 2019.

[21] Y. Lee, A. Szot, S.-H. Sun, and J. J. Lim, "Generalizable imitation learning from observation via inferring goal proximity," in *Advances in Neural Information Processing Systems*, vol. 34, 2021.

[22] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Advances in Neural Information Processing Systems*, 2016, pp. 4565–4573.

[23] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, 2014, pp. 2672–2680.

[24] Z. Sui, L. Xiang, O. C. Jenkins, and K. Desingh, "Goal-directed robot manipulation through axiomatic scene estimation," *The International Journal of Robotics Research*, vol. 36, no. 1, pp. 86–104, 2017.

[25] Z. Zeng, Z. Zhou, Z. Sui, and O. C. Jenkins, "Semantic robot programming for goal-directed manipulation in cluttered scenes," in *International Conference on Robotics and Automation*, 2018, pp. 7462–7469.

[26] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, "Learning latent plans from play," *arXiv preprint arXiv:1903.01973*, 2019.

[27] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *International Conference on Robotics and Automation*, 2018, pp. 4693–4700.

[28] A. L. Friesen and R. P. Rao, "Imitation learning with hierarchical actions," in *2010 IEEE 9th International conference on development and learning*. IEEE, 2010, pp. 263–268.

[29] H. Dindo and G. Schillaci, "An adaptive probabilistic approach to goal-level imitation learning," in *International Conference on Intelligent Robots and Systems*, 2010, pp. 4452–4457.

[30] A. Mandlekar, F. Ramos, B. Boots, S. Savarese, L. Fei-Fei, A. Garg, and D. Fox, "Iris: Implicit reinforcement without interaction at scale for learning control from offline robot manipulation data," in *International Conference on Robotics and Automation*, 2020, pp. 4414–4420.

[31] A. Mandlekar, D. Xu, R. Martín-Martín, S. Savarese, and L. Fei-Fei, "Learning to generalize across long-horizon tasks from human demonstrations," *Robotics: Science and Systems*, 2020.

[32] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[33] D. G. Caldwell, S. Davis, R. J. Moreno Masey, and J. O. Gray, "Automation in food processing," in *Springer Handbook of Automation*. Springer, 2009, pp. 1041–1059.

[34] M. Wilson, "Developments in robot applications for food manufacturing," *Industrial Robot: An International Journal*, 2010.

[35] C. Wang, T. Luo, L. Zhao, Y. Tang, and X. Zou, "Window zooming–based localization algorithm of fruit and vegetable for harvesting robot," *IEEE Access*, vol. 7, pp. 103 639–103 649, 2019.

[36] L. Zhang, G. Gui, A. M. Khattak, M. Wang, W. Gao, and J. Jia, "Multi-task cascaded convolutional networks based intelligent fruit detection for designing automated robot," *IEEE Access*, vol. 7, pp. 56 028–56 038, 2019.

[37] Y. Yu, K. Zhang, L. Yang, and D. Zhang, "Fruit detection for strawberry harvesting robot in non-structural environment based on mask-rcnn," *Computers and Electronics in Agriculture*, vol. 163, p. 104846, 2019.

[38] A. Kuznetsova, T. Maleva, and V. Soloviev, "Using yolov3 algorithm with pre-and post-processing for apple detection in fruit-harvesting robot," *Agronomy*, vol. 10, no. 7, p. 1016, 2020.

[39] C. Huangfei *et al.*, "Design and research on the end actuator of tomato picking robot," *Journal of Physics: Conference Series*, vol. 1314, no. 1, p. 012112, 2019.

[40] L. Mu, G. Cui, Y. Liu, Y. Cui, L. Fu, and Y. Gejima, "Design and

simulation of an integrated end-effector for picking kiwifruit by robot," *Information Processing in Agriculture*, vol. 7, no. 1, pp. 58–71, 2020.
[41] M. Bollini, S. Tellex, T. Thompson, N. Roy, and D. Rus, "Interpreting and executing recipes with a cooking robot," in *Experimental Robotics*, 2013, pp. 481–495.
[42] H. Zhang and S. Nikolaidis, "Robot learning and execution of collaborative manipulation plans from youtube cooking videos," *arXiv preprint arXiv:1911.10686*, 2019.
[43] S. Davis, J. W. Casson, R. J. M. Masey, M. King, J. O. Gray, and D. G. Caldwell, "Robot prototyping in the design of food processing machinery," *Industrial Robot: An International Journal*, 2007.
[44] S. Ma, L. Du, E. Tsuchiya, and M. Fuchimi, "made grippers for soft food grasping," in *International Conference on Ubiquitous Robots*, 2020, pp. 362–367.
[45] X. Lin, Z. Huang, Y. Li, J. B. Tenenbaum, D. Held, and C. Gan, "Diffskill: Skill abstraction from differentiable physics for deformable object manipulations with tools," *arXiv preprint arXiv:2203.17275*, 2022.
[46] X. Lin, C. Qi, Y. Zhang, Z. Huang, K. Fragkiadaki, Y. Li, C. Gan, and D. Held, "Planning with spatial-temporal abstraction from point clouds for deformable object manipulation," in *6th Annual Conference on Robot Learning*, 2022.
[47] M. A. Lee, C. Florensa, J. Tremblay, N. Ratliff, A. Garg, F. Ramos, and D. Fox, "Guided uncertainty-aware policy optimization: Combining learning and model-based strategies for sample-efficient policy learning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 7505–7512.
[48] J. Yamada, Y. Lee, G. Salhotra, K. Pertsch, M. Pflueger, G. Sukhatme, J. Lim, and P. Englert, "Motion planner augmented reinforcement learning for robot manipulation in obstructed environments," in *Conference on Robot Learning*. PMLR, 2021, pp. 589–603.
[49] Y. Liu, A. Gupta, P. Abbeel, and S. Levine, "Imitation from observation: Learning to imitate behaviors from raw video via context translation," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 1118–1125.
[50] L. Smith, N. Dhawan, M. Zhang, P. Abbeel, and S. Levine, "Avid: Learning multi-stage tasks via pixel-level translation of human videos," *arXiv preprint arXiv:1912.04443*, 2019.
[51] J. Yang, J. Zhang, C. Settle, A. Rai, R. Antonova, and J. Bohg, "Learning periodic tasks from human demonstrations," *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
[52] H. Kim, Y. Ohmura, A. Nagakubo, and Y. Kuniyoshi, "Training robots without robots: Deep imitation learning for master-to-robot policy transfer," *arXiv preprint arXiv:2202.09574*, 2022.
[53] D. Chotrov, Z. Uzunova, Y. Yordanov, and S. Maleshkov, "Mixed-reality spatial configuration with a zed mini stereoscopic camera," 11 2018.
[54] J. Paillard, "Fast and slow feedback loops for the visual correction of spatial errors in a pointing task: a reappraisal," *Canadian Journal of Physiology and Pharmacology*, vol. 74, no. 4, pp. 401–417, 1996.
[55] A. Billard, S. Calinon, R. Dillmann, and S. Schaal, "Robot programming by demonstration," *Springer Handbook of Robotics*, pp. 1371–1394, 2008.
[56] L. Bazzani, H. Larochelle, and L. Torresani, "Recurrent mixture density network for spatiotemporal visual attention," in *International Conference on Learning Representations*, 2016.
[57] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
[58] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Neural Information Processing Systems*, 2017, pp. 5998–6008.
[59] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
[60] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4190–4197.
[61] Y. Kuniyoshi, M. Inaba, and H. Inoue, "Learning by Watching: Extracting reusable task knowledge from visual observation of human performance," *Transactions on Robotics and Automation*, vol. 10, no. 6, pp. 799–822, 1994.
[62] G. Konidaris, S. Kuindersma, R. Grupen, and A. Barto, "Robot learning from demonstration by constructing skill trees," *The International Journal of Robotics Research*, vol. 31, no. 3, pp. 360–375, 2012.
[63] C. Yu and D. H. Ballard, "Understanding human behaviors based on eye-head-hand coordination," in *International Workshop on Biologically Motivated Computer Vision*, 2002, pp. 611–619.
[64] J. Pelz, M. Hayhoe, and R. Loeber, "The coordination of eye, head, and hand movements in a natural task," *Experimental Brain Research*, vol. 139, pp. 266–77, 2001.
[65] M. Hayhoe and D. Ballard, "Eye movements in natural behavior," *Trends in Cognitive Sciences*, vol. 9, pp. 188–94, 2005.
[66] H. Kim, Y. Ohmura, and Y. Kuniyoshi, "Memory-based gaze prediction in deep imitation learning for robot manipulation," *arXiv preprint arXiv:2202.04877*, 2022.
[67] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han, "On the variance of the adaptive learning rate and beyond," in *International Conference on Learning Representations*, 2020, pp. 1–14.
[68] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," in *AAAI conference on artificial intelligence*, vol. 34, no. 07, 2020, pp. 13 001–13 008.